

Công cụ tìm tài liệu có ràng buộc thời gian

RETECO SemEval-2027 · Track 1a · Temporal Grounded Retrieval

1.654.055
TÀI LIỆU

1.211
CÂU TRUY VẤN

0,1890
NDCG@10 HIỆN TẠI

2,63×
SO VỚI CƠ SỞ

13
NHÓM LĨNH VỰC

4,12
GB VĂN BẢN

29,4%
LÀ BẢN SAO

3,57
ĐÁP ÁN MỖI CÂU

99,3%
TRONG TẦM TỪ KHOẢ

0,5718
TRẦN HIỆN TẠI

ĐÃ CHẠY XONG

10 / 15 notebook

Từ 00 tới 06b : số liệu có thật. Từ 07a tới 08 : code đã viết, đang thực nghiệm.

NỘI DUNG

1	Bài toán	2	2	Kết quả và lộ trình	2
3	Cách kiểm chứng	4	4	Dữ liệu	4
5	Chọn term	6	6	Trần và khoảng trống	6
7	Mô hình truy xuất	7	8	Công thức xếp hạng	8
9	Chỉ mục	8	10	Đánh giá đầy đủ	9
11	Khử trùng lặp	9	12	Đang thực nghiệm	11
13	Thư mục <code>src/</code>	12	14	Thuật ngữ	12
15	Giả thuyết bị bác bỏ	14			

CÁCH ĐỌC BÁO CÁO NÀY

1 Mục 1 tới 3
Bài toán là gì, đang ở đâu, và làm sao biết một cái tiến là thật

2 Mục 4 tới 11
Từng thí nghiệm: hỏi gì, đo được gì, vì sao kết quả như vậy

3 Mục 12 tới 15
Phần đang làm dở, thư mục mã nguồn, thuật ngữ, và những giả thuyết đã sai

Mọi con số đều là nDCG@10 trung bình theo 13 nhóm, trên tập `train`, trừ khi ghi rõ khác.

1 Bài toán

Kho văn bản rất lớn. Người dùng gõ một câu hỏi. Máy trả về **10 tài liệu**, xếp cái đúng nhất lên đầu.

Khác công cụ tìm kiếm thường ở một điểm: câu hỏi mang **ràng buộc thời gian**.

“Tính đến năm 2017 thì tình hình thế nào?”

ĐÚNG đúng chủ đề · mô tả **năm 2017**

SAI đúng chủ đề · mô tả **năm 2021**

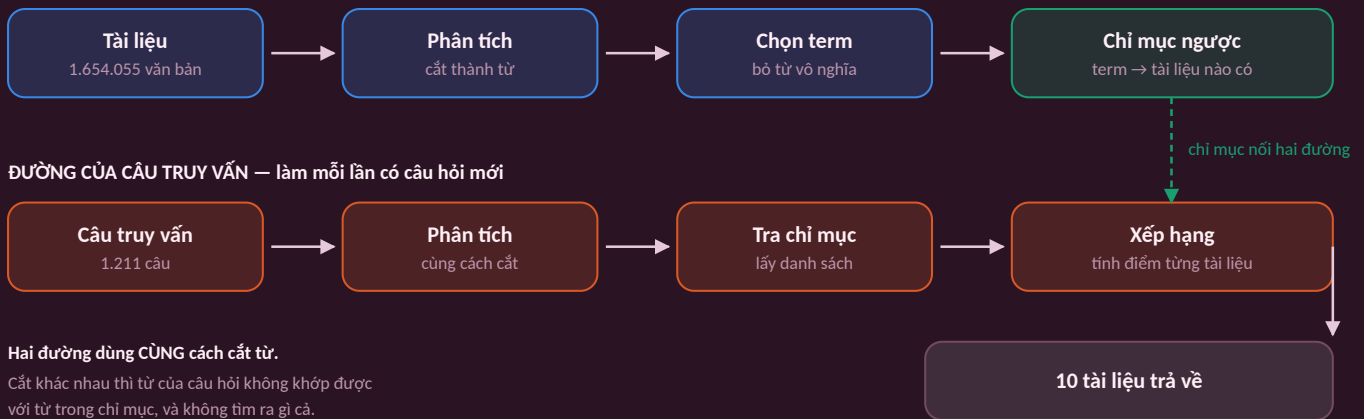
Tài liệu thứ hai dùng đúng từ ngữ nhưng sai thời điểm, vẫn bị tính là sai.

CHẤM ĐIỂM

Chỉ số	nDCG@10
Gộp theo	trung bình 13 nhóm
Nhân đáp án	nhị phân, mọi giá trị = 1
Công cụ chấm	pytrec_eval · ndcg_cut_10
Mốc so sánh	BM25 ban tổ chức, cùng tập

Bẫy của cách gộp. Nhóm `history` giữ 561 trong 1.211 câu (46%) nhưng chỉ chiếm **7,7%** điểm, vì trung bình tính theo nhóm. Làm tốt riêng nhóm lớn nhất gần như vô ích.

ĐƯỜNG CỦA TÀI LIỆU — làm một lần, trước khi có câu hỏi



Hai đường dùng **CÙNG** cách cắt từ.

Cắt khác nhau thì từ của câu hỏi không khớp được với từ trong chỉ mục, và không tìm ra gì cả.

2 Kết quả và lộ trình

MỐC	NDCG@10	×	NGUỒN
Hệ thống cơ sở	0,0719	1,00	02
BM25 ban tổ chức	0,0879	1,22	starter kit
Sau khi chọn term	0,1463	2,03	03a
Query Likelihood	0,1839	2,56	04
Tinh chỉnh công thức	0,1890	2,63	05
Trần nếu xếp hạng lại	0,5718	7,95	03b

Cột $\times$ so với hệ thống cơ sở. So với BM25 ban tổ chức, cùng tập `train`, hiện tại đạt **2,15 lần**.

KHOẢNG CÁCH CÒN LẠI

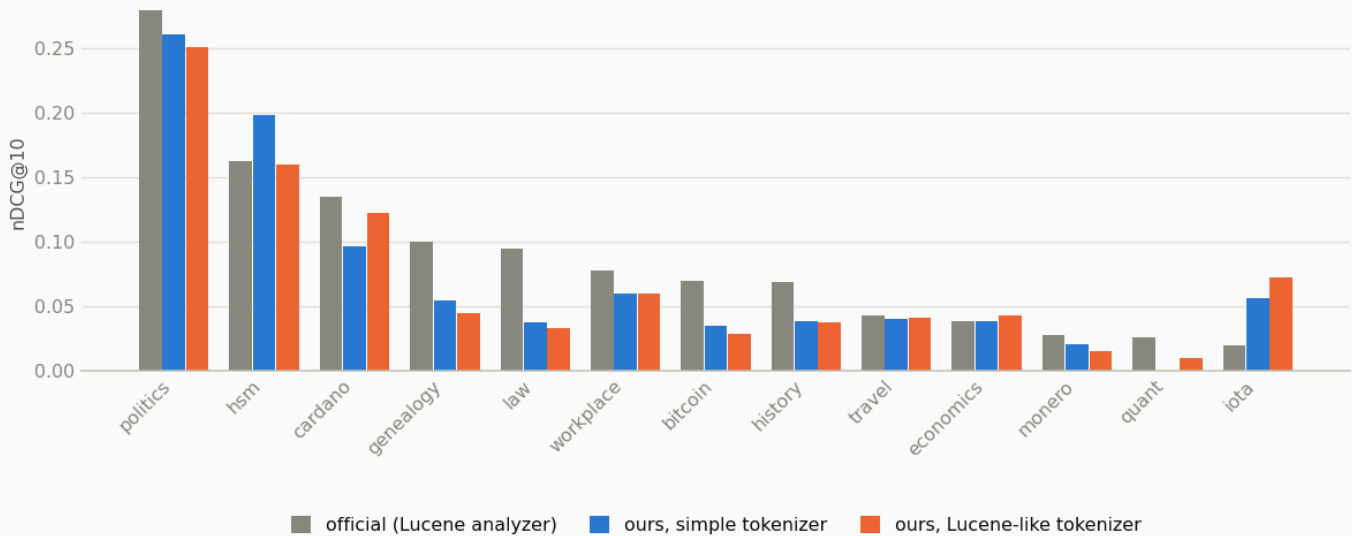
0,1890 hiện tại 0,5718 trần

Trần là điểm cao nhất đạt được nếu sắp xếp lại **hoàn hảo** đúng danh sách 10 ứng viên đang có. Phần màu nhạt là chỗ còn trống.

Đáp án phần lớn **đã tìm thấy**, chỉ chưa xếp lên top 10. Đây là bài toán **xếp hạng**, không phải bài toán tìm kiếm.

Official baseline against two tokenisations of our own

nDCG@10 per domain, train split. Same BM25 formula in all three.



Mốc chính thức của ban tổ chức so với hai cách cắt từ của dự án, theo từng nhóm — cùng một công thức BM25 cho cả ba

Hệ quy chiếu

Một điểm số chỉ có nghĩa khi kèm theo tập câu hỏi đã chấm nó. Có ba tập, cùng một độ đo, và số của tập này **không so thẳng** được với tập kia.

TẬP	SỐ CÂU	AI CHẤM
train	1.211	tự chấm, không giới hạn
dev	519	tự chấm, một lần
test ẩn	chưa có	ban tổ chức

Chưa có bảng xếp hạng. Dữ liệu phát hành 30/08/2026; mục tin tức của ban tổ chức ghi phần tiếp theo là đăng ký và nền tảng thi đấu. Chưa ai nộp gì, nên **không có căn cứ** để nói bao nhiêu điểm là cao. Mốc duy nhất công bố được là BM25 của chính ban tổ chức.

Vì vậy dự án không đặt mục tiêu theo một điểm số tuyệt đối, mà theo ba đại lượng đo được:

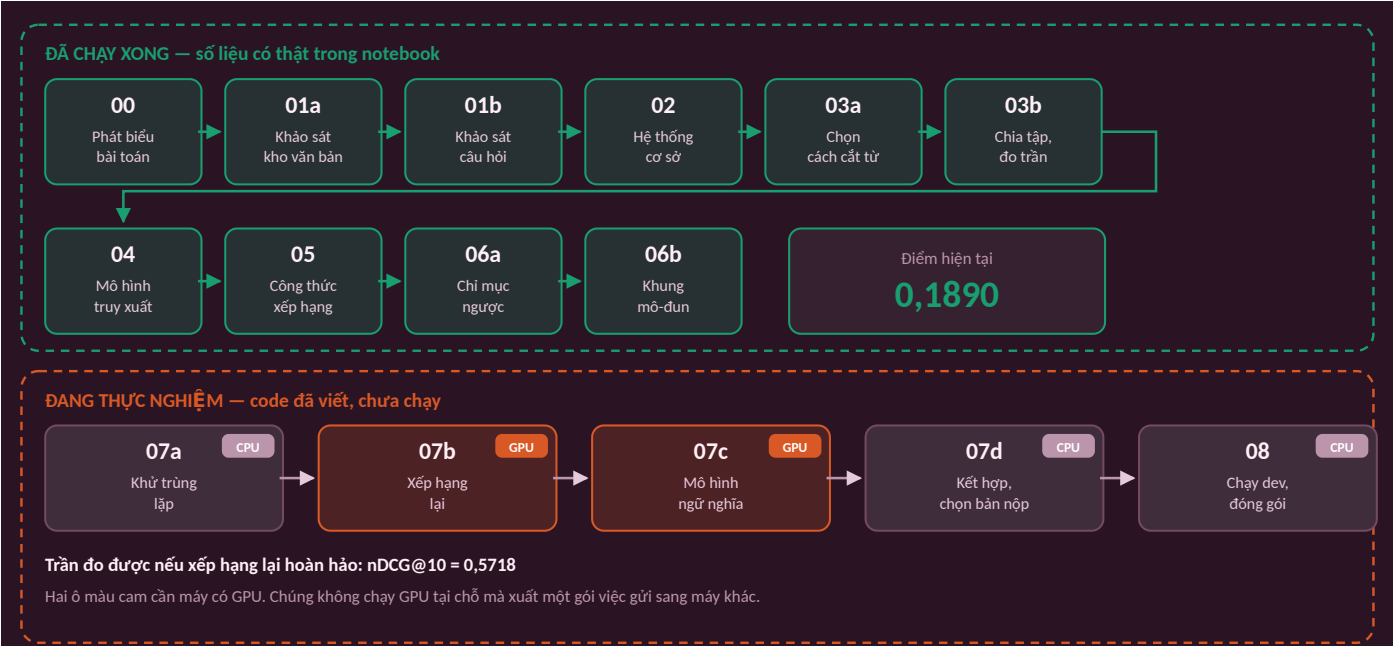
Tỷ lệ so với BM25 ban tổ chức, cùng train	2,12×
Số nhóm thắng mốc chính thức	12 / 13
Phần khoảng trống tới trần@100 đã thu	23,0%

BM25 ban tổ chức so với `bm25_tuned` của dự án, cả hai trên `train`, cùng cách gộp theo nhóm.

Nhóm	BAN TỔ CHỨC	DỰ ÁN	×
iota	0,0199	0,2656	13,35
quant	0,0255	0,1436	5,63
monero	0,0278	0,1078	3,88
economics	0,0382	0,1339	3,51
travel	0,0429	0,1432	3,34
history	0,0691	0,2046	2,96
workplace	0,0777	0,2126	2,74
genealogy	0,1003	0,2169	2,16
law	0,0943	0,1996	2,12
bitcoin	0,0695	0,1318	1,90
cardano	0,1349	0,2181	1,62
hsm	0,1627	0,2133	1,31
politics	0,2792	0,2355	0,84
Trung bình nhóm	0,0878	0,1867	2,12

Thắng **12 trên 13** nhóm. Nhóm thua duy nhất là `politics`, cũng là nhóm mốc chính thức mạnh nhất.

Ước lượng trên dev. Chính BM25 của ban tổ chức chạy cao hơn 10% trên `dev` so với `train` (0,0967 so với 0,0879). Nếu hệ thống này giữ cùng tỷ lệ đó, 0,1890 trên `train` tương ứng khoảng **0,21 trên dev**. Đây là ngoại suy; notebook 08 sẽ đo thật, một lần.

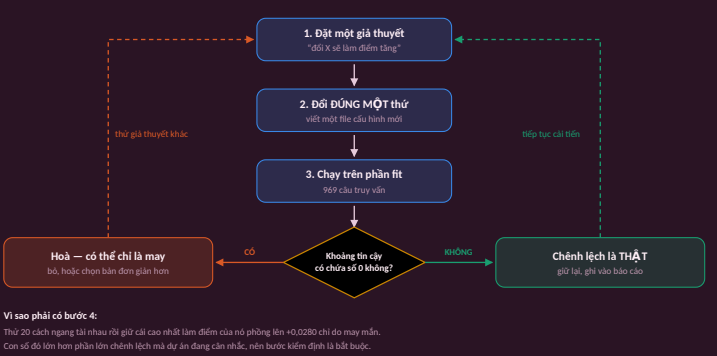


3 Cách kiểm chứng một cải tiến

Vì sao cần bước kiểm định. Notebook 03b mô phỏng: tạo 20 hệ thống ngang tài nhau, khác biệt chỉ do ngẫu nhiên, rồi giữ cái cao nhất. Cái được giữ có điểm cao hơn mức thật +0,0280 — chỉ vì thử 20 lần thì có lần gặp may.

Con số đó lớn hơn phần lớn chênh lệch dự án đang cân nhắc, nên mọi kết luận đều phải qua bootstrap.

KHOẢNG TIN CẬY	KẾT LUẬN
không chứa 0	chênh lệch THẬT
chứa 0	hoà — chọn bản đơn giản hơn



1.211 câu truy vấn của tập train

fit — 969 câu (80%) dùng để CHỌN	check — 242 chỉ XÁC NHẬN	dev — 519 câu CHẠY ĐÚNG 1 LẦN
--	------------------------------------	---

Ba phần, ba vai trò khác nhau

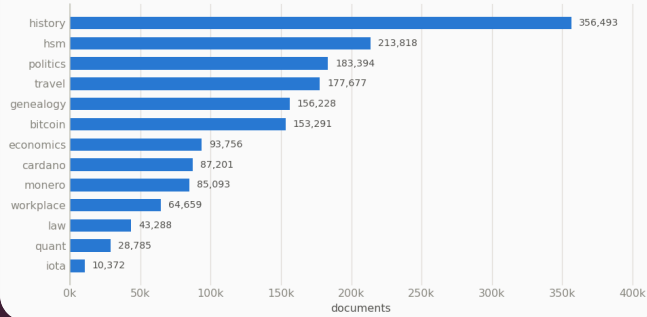
- fit** Thử bao nhiêu lần cũng được. Mọi lựa chọn đều quyết định trên phần này.
- check** Đọc SAU KHI đã chọn, để xem lựa chọn có bị khớp riêng vào fit không.
- dev** Chạy một lần ở cuối. Dừng lại lần hai là nó thành tập train thứ hai.

4 Dữ liệu

1.654.055 TÀI LIỆU	34× NHÓM LỚN / NHỎ	241	29,4% LÀ BẢN SAO	3,57 ĐÁP ÁN MỖI CÂU	100% CÂU HỎI CÓ HTML
-----------------------	-----------------------	-----	---------------------	------------------------	-------------------------

Corpus size varies 34x across domains

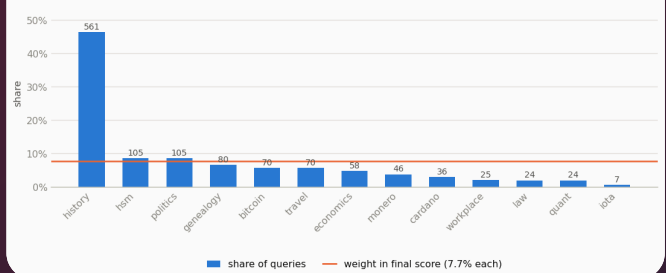
Number of documents per domain



Nhóm lớn nhất gấp nhỏ nhất 34 lần — nhưng mỗi nhóm vẫn chiếm 1/13 điểm

Query counts are lopsided; scoring weight is not

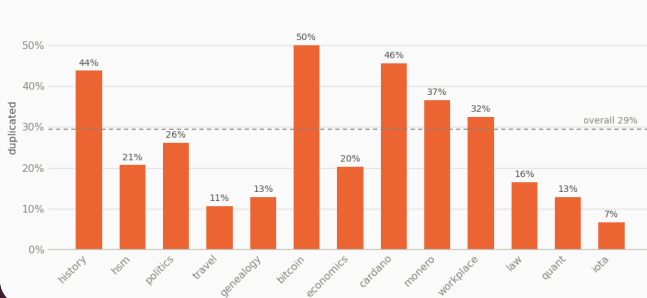
Bars: share of all train queries. Line: share of the final score.



history giữ 46% câu hỏi, chỉ nhận 7,7% trọng số

Roughly 29% of the corpus is duplicated text

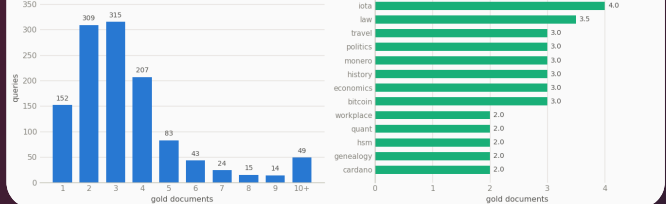
Share of documents whose content is an exact copy of another



29% kho là bản sao · **bitcoin** 50%, **cardano** 46%

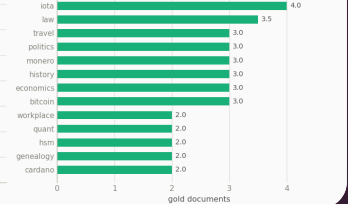
Most queries have two to four answers

Gold documents per query (values above 9 pooled)



Stable across domains

Median gold documents



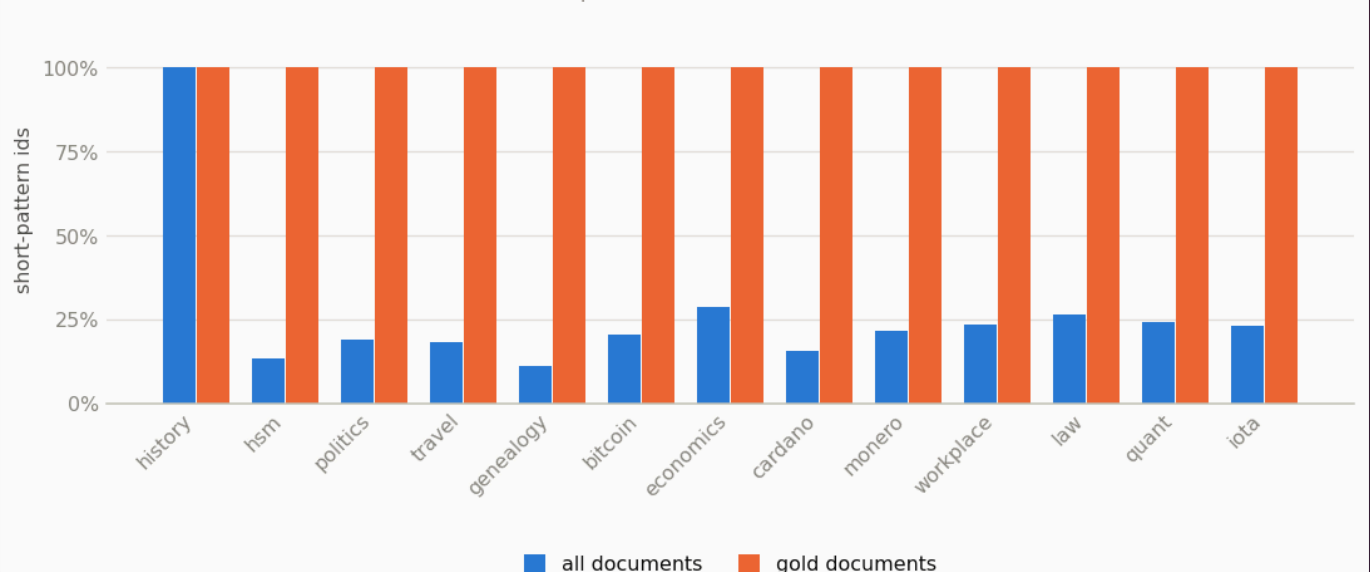
Phần lớn câu hỏi có 2-4 đáp án — trần của P@10 chỉ khoảng 0,36

Rò rỉ mã tài liệu — ghi lại nhưng cố tình KHÔNG dùng. Mọi tài liệu là đáp án đều mang mã theo một khuôn dạng riêng, trong khi khuôn dạng đó chỉ chiếm **11–26%** kho. Lọc theo mã loại được **75–89%** kho mà không mất đáp án nào.

Không đưa vào hệ thống dự thi vì: nó khai thác cách bộ dữ liệu được tạo ra chứ không giải bài toán; không có gì bảo đảm còn đúng trên tập test ẩn; điểm thu được không phản ánh chất lượng thật.

Every gold document carries a 'short' id

Share of documents whose id follows the short pattern



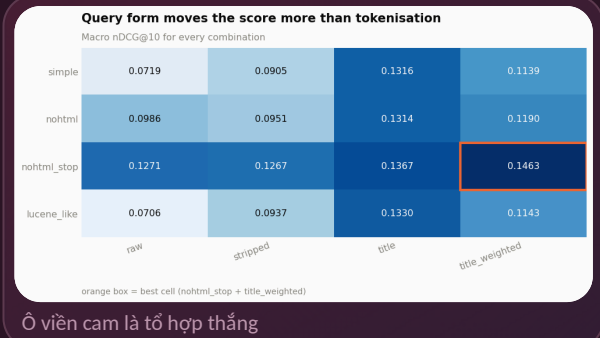
Cột cam là tài liệu đáp án, cột xanh là toàn kho — mọi nhóm đều lệch hẳn

5 Chọn term

03A

Thứ 4 cách cắt từ × 4 dạng câu hỏi. Điểm tăng từ **0,0719** lên **0,1463** — hơn gấp đôi, và đây là bước tăng điểm lớn nhất của cả dự án.

	RAW	BỎ HTML	TIÊU ĐỀ	TIÊU ĐỀ ×3
đơn giản	0,0719	0,0905	0,1316	0,1139
bỏ chữ HTML	0,0986	0,0951	0,1314	0,1190
+ bỏ hư từ	0,1271	0,1267	0,1367	0,1463
kiểu Lucene	0,0706	0,0937	0,1330	0,1143



Dạng câu hỏi dịch chuyển điểm nhiều hơn cách cắt từ: chênh lệch theo hàng ngang lớn hơn theo cột dọc.

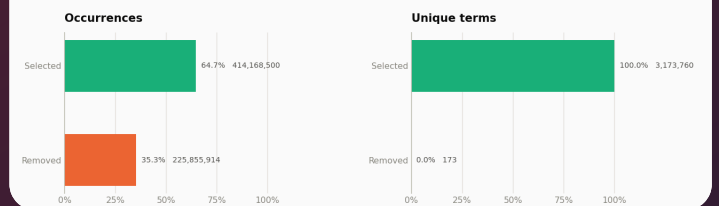
TỶ LỆ TERM ĐƯỢC CHỌN · QUÉT TOÀN KHO 7,7 PHÚT

Lượt xuất hiện	640.024.414 → 414.168.500	64,7%
Term khác nhau	3.173.933 → 3.173.760	99,99%

Danh sách loại trừ chỉ **173 từ** nên gần như không đụng tới từ vựng, nhưng chúng lặp ở khắp nơi nên gom về **35,3%** tổng số lượt. Riêng **the** chiếm **5,27%** kho.

Term selection removes 35% of occurrences, retaining 100.0% of the vocabulary

173 terms in the exclusion list, across 1,654,055 documents



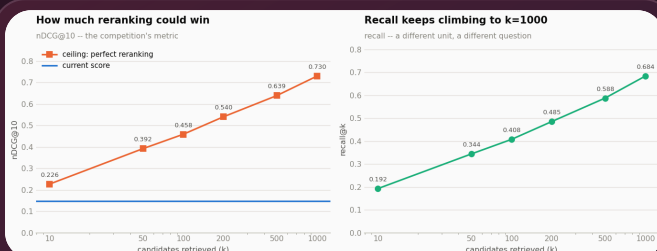
Bỏ 35% số lượt, giữ gần như toàn bộ từ vựng

Điều lọt ra ngoài dự kiến. 20 term sống sót đứng đầu gần như không có từ nào mang nghĩa: **s** (4.432.973 lượt), chữ số đứng một mình, **url**, **t**, **n**, **c**. Đó là mảnh vụn do cách cắt tách ở dấu nhảy. Công tắc **min_length** xử lý được, nhưng chưa đo nên chưa đưa vào.

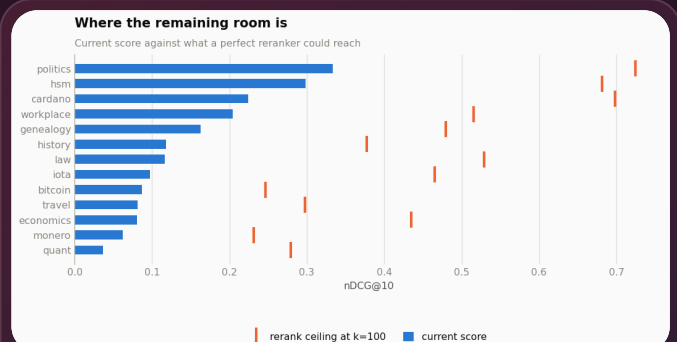
6 Trần và khoảng trống

03B

Điểm lúc đo	0,1463	
Trần nếu xếp hạng lại 100 ứng viên	0,4584	còn rất nhiều đất
Trần nếu xếp hạng lại 1.000 ứng viên	0,7296	lấy sâu hơn thì trần cao hơn nhiều
recall@100 → recall@1000	0,4075 → 0,6839	
Đáp án có từ chung với câu hỏi	99,3%	4.351 / 4.381



Trần leo đều tới k=1000 — nâng độ sâu gần như miễn phí



Khoảng trống theo từng nhóm: vạch cam là trần, cột xanh là điểm hiện tại

Con số 99,3% định hướng cả dự án. Chỉ 0,7% đáp án là thứ cách tìm theo từ khoá về nguyên tắc không với tới. Nên mô hình hiểu ngữ nghĩa (đắt, cần GPU) có trần rất thấp, còn cải thiện cách xếp hạng thì còn rất nhiều đất. Đó là lý do xếp hạng lại đứng **trước** mô hình ngữ nghĩa trong kế hoạch.

7 Mô hình truy xuất

04

MÔ HÌNH	HƯỚNG	NDCG@10	R@100	TRẦN	TRẢ LỜI
Query Likelihood $\mu=1000$	xác suất · uni-gram	0,1839	0,4996	0,5571	1.211
BM25 $k1=0,9$ $b=0,4$	xác suất · BIM	0,1463	0,4075	0,4584	1.211
TF-IDF	không gian vector	0,1407	0,5084	0,5680	1.211
TF (bỏ idf)	không gian vector	0,0021	0,0197	0,0235	1.211
Boolean OR	tập hợp	0,0014	0,0103	0,0125	1.211
Boolean AND	tập hợp	0,0041 bẫy	0,0030	0,0041	141

SỐ SÁNH	CHÊNH	KHOẢNG TIN CẬY	
qlm vs bm25	+0,0376	[+0,0234, +0,0530]	THẬT
qlm vs tfidf	+0,0432	[+0,0266, +0,0594]	THẬT
bm25 vs tfidf	+0,0055	[-0,0183, +0,0271]	hoà

Query Likelihood thắng cả hai một cách có ý nghĩa — lần đầu một phép so ở notebook này cho kết quả không hoà. Còn BM25 với TF-IDF thì hoà, và TF-IDF lật ngược nếu đổi cách cắt từ.



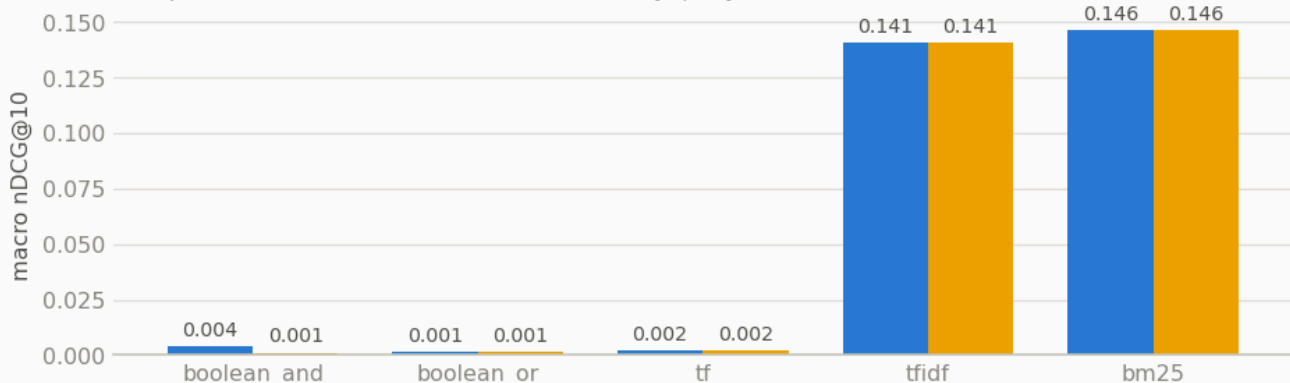
Quét μ : điểm chỉ đổi 0,0327 trên cả dải · bốn mô hình trên phần fit

Cái bẫy của bộ chấm điểm. `boolean_and` chỉ trả lời **141/1.211** câu. Bộ chấm chính thức chỉ chấm những câu có trong file kết quả — câu không trả lời được thì bị **bỏ qua**, không phải cho 0 điểm.

Hệ quả: hệ thống chỉ trả lời câu nào nó chắc sẽ *trông* cao điểm hơn hệ thống trả lời hết. Điểm của nó bị thổi phồng **3,7 lần**. Vì vậy mọi bảng trong dự án đều có cột “trả lời được”.

`boolean_and` answers 141 of 1,211 queries, and the scorer ignores the rest

its reported score is 3.7x what it earns over every query



Two bars are equal for any model that returns something for every query.
This is why every score in this project is reported with num_topics.

Cột vàng là điểm nếu câu không trả lời được bị tính 0 — đó mới là con số trung thực

8 Công thức xếp hạng

05

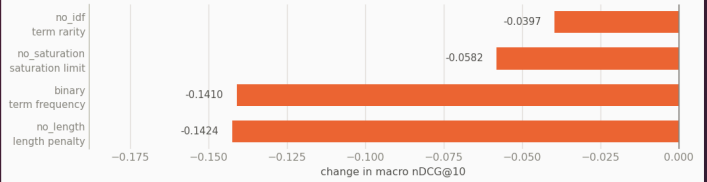
Gỡ từng thành phần ra khỏi công thức rồi đo xem mất bao nhiêu.

GỠ CÁI GÌ	CÁCH GỠ	CÒN	MẤT
(nguyên bản)	—	0,1463	—
Trừng phạt độ dài	$b=0$	0,0039	-0,1424
Vai trò lặp từ	$k1 \rightarrow 0$	0,0053	-0,1410
Giới hạn lặp từ	$k1 \rightarrow \infty$	0,0881	-0,0582
Độ hiếm của từ	bỏ idf	0,1066	-0,0397

Chuẩn hoá độ dài quan trọng gấp 5 lần idf . Kho có tài liệu dài hàng trăm nghìn từ; một tài liệu dài chứa gần như mọi từ, nên tất phần trừng phạt là danh sách bị chúng chiếm hết.

Removing length penalty costs the most

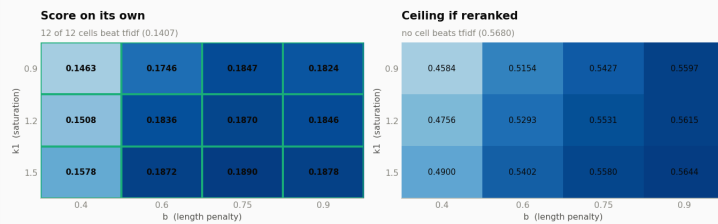
change in macro nDCG@10 against full BM25 at 0.1463



13 domains, train split. Each bar removes exactly one component and leaves the rest of the formula untouched.

Bỏ trừng phạt độ dài tổn nhiều nhất

No (k1, b) lifts BM25's ceiling to tfidf's



13 domains, train split, top-100 retrieved. Cells outlined in green beat the BM25 reference. The two panels measure different things and are read separately.

Lưới $k1 \times b$ — trái là điểm hiện tại, phải là trần

CHỌN THEO ĐIỂM

$k1=1,5$ $b=0,75$

nDCG 0,1890 · trần
0,5580

CHỌN THEO TRẦN

$k1=2,0$ $b=0,9$

nDCG 0,1867 · trần
0,5718

Dự án chọn cấu hình bên phải, tuy điểm thấp hơn 0,0023. Lý do: bước sau là xếp hạng lại, và nó chỉ sắp xếp lại những gì tăng một đưa cho. Tăng một tốt là tăng một tìm được nhiều đáp án, không phải tăng một xếp đúng.

Cêngh so với mặc định: +0,0427, khoảng tin cậy [+0,0287, +0,0579] — thật.

9 Chỉ mục ngược

06A

216.320.198

CẬP (TỪ, TÀI LIỆU)

2,63

GB CHỈ MỤC

13,6

PHÚT DỰNG

2.158×

NHANH HƠN QUÉT

14

PHÚT CẢ TẬP
TRAIN

43

GIỜ NẾU KHÔNG
CÓ

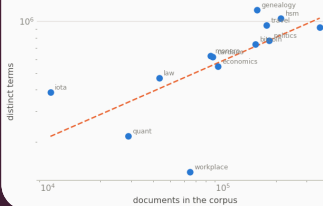
Chỉ mục ngược lật ngược bảng: thay vì “tài liệu này chứa những từ nào”, nó lưu “từ này nằm trong những tài liệu nào”.

Nhờ đó chỉ cần duyệt tới tài liệu có chứa từ của câu hỏi, thay vì quét cả kho. Con số 43 giờ so với 14 phút là lý do phần này tồn tại — không có chỉ mục thì không thí nghiệm được gì cả.

Từ vưng tăng theo $(số\ tài\ liệu)^{0,44}$: kho to gấp đôi thì từ vưng chỉ tăng khoảng 36%.

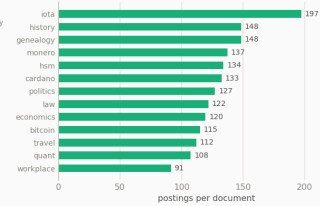
Vocabulary grows as documents^{0.44}

each point is one domain; both axes logarithmic



Distinct terms per document

how many cells one document adds to the index



Từ vưng tăng chậm hơn số tài liệu · số posting mỗi tài liệu

10 Đánh giá đầy đủ

06B

HỆ THỐNG	P@10	R@10	F1@10	MAP	NDCG@10	R@100
Đã tinh chỉnh	0,0632	0,2371	0,0957	0,1469	0,1867	0,5125
Mặc định	0,0496	0,1923	0,0754	0,1150	0,1463	0,4075
chênh	+0,0136	+0,0448	+0,0203	+0,0319	+0,0404	+0,1050

Cả sáu độ đo xếp cùng một thứ tự. Khi các độ đo đồng thuận thì kết luận không phụ thuộc vào việc chọn độ đo nào. **P@10 thấp là bình thường:** mỗi câu chỉ có trung bình 3,57 đáp án nên trong 10 ô nhiều nhất cũng chỉ 3-4 ô có thể đúng, trần của P@10 vào khoảng 0,36.

1.211 câu truy vấn, chia theo “cần gì để sửa”



Hai nhóm bên phải cần hai cách chữa khác hẳn nhau

- Bước xếp hạng lại sửa được — đáp án đã nằm trong danh sách, chỉ cần kéo lên.
- Xếp hạng lại KHÔNG cứu được — phải lấy danh sách sâu hơn, hoặc đổi tăng một.

29,8% là toàn bộ phần bước xếp hạng lại có thể giành được ở độ sâu 100 — biết trước trần của nó trước khi bỏ công làm.

23,0% mới là chỗ làm đổi kế hoạch. Gần một phần tư số câu có đáp án *không hề nằm trong danh sách ứng viên*. Vì vậy **07b** làm **hai** việc: nâng độ sâu lên 1.000 trước, rồi mới xếp hạng lại.

PHÂN TÍCH CAO VÀ THẤP

- 5 câu cao nhất nDCG **1,000** · 1-3 đáp án · đáp án ở hạng 1
- 5 câu thấp nhất 3 câu đáp án **ngoài 100 ứng viên**
- 2 câu đáp án ở **hạng 22 và 31**

11 Khử trùng lặp

07A

29,4%

KHO LÀ BẢN SAO

26,4%

CHỖ ỨNG VIÊN BỊ CHIẾM

51,0%

RIÊNG BITCOIN

47

ĐÁP ÁN CÓ BẢN SAO KÈM

2

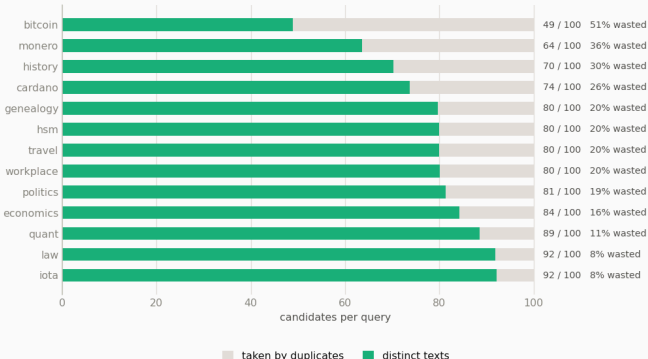
GIÁ TRỊ EXPAND ĐÃ CHỐT

2.043

ĐỘ SÂU 07B CẦN LẤY

On average 26% of every candidate list is duplicate text

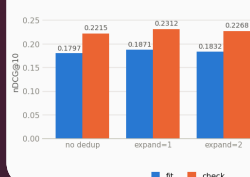
bm25_tuned, depth 100, train split



Phần xám là chỗ bị bản sao chiếm — bitcoin mất một nửa danh sách

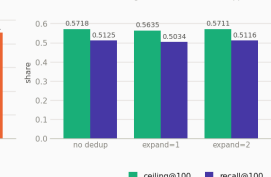
The score barely moves with expand

Chosen on fit, confirmed on check



How many answers collapsing loses

A lower recall means gold documents were dropped



Điểm đổi rất ít theo expand · nhưng recall thì không

CẤU HÌNH	FIT	NDCG	R@100	TRẦN
không gộp	0,1797	0,1867	0,5125	0,5718
expand=1	0,1871	0,1944	0,5034	0,5635
expand=2 đã chọn	0,1832	0,1905	0,5116	0,5711
expand=3	0,1811	0,1881	0,5116	0,5711

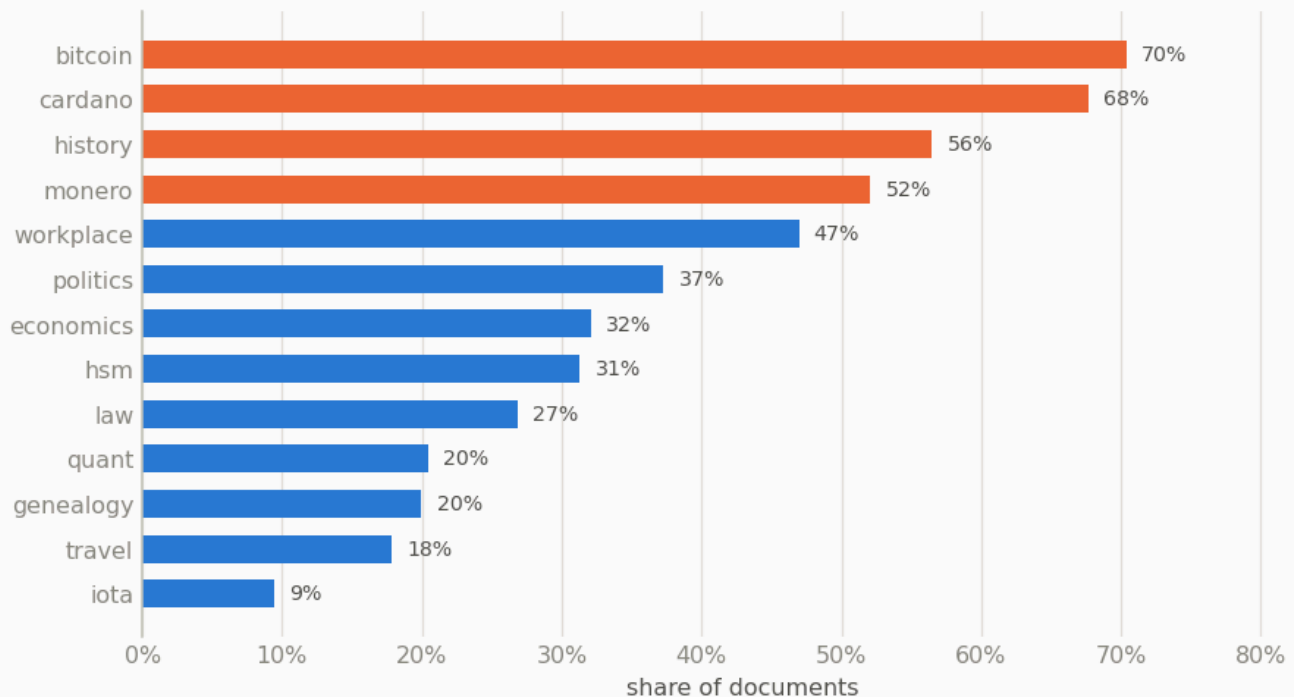
Vì sao không chọn **expand=1** dù nó điểm cao nhất. Nó mất **0,0090 recall**. Vì **07b** sẽ xếp hạng lại, recall là chặn trên của những gì bước đó với tới được — mất 0,0090 recall đắt hơn được 0,0039 nDCG.

Đây đúng là logic của **05** khi chọn **k1=2,0 b=0,9** : tăng một chọn theo **trần**, không theo điểm hiện tại.

Cái bẫy: danh sách đáp án không đánh dấu mọi bản sao. Có **47** đáp án nằm cùng danh sách với bản sao của chính nó, nên gộp ngay thơ là điểm 0 cho một văn bản đúng từng chữ.

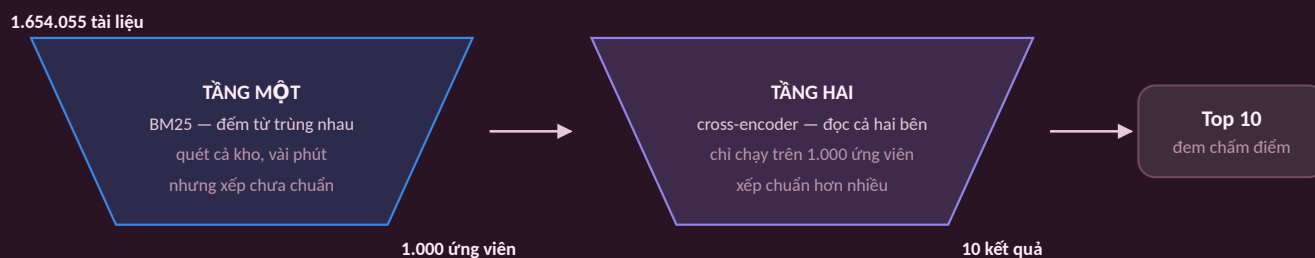
4 domains hold over half their documents in a duplicate group

Share of documents with at least one twin in the same domain



Bốn nhóm có quá nửa tài liệu nằm trong một nhóm bản sao — **bitcoin** 70%, **cardano** 68%, **history** 56%, **monero** 52%

Năm notebook đã viết xong và chạy thử trên bộ dữ liệu giả, nhưng **chưa chạy trên dữ liệu thật**. Phần này mô tả chúng định đo cái gì. Chưa có số liệu.



Vì sao phải chia làm hai tầng

Cross-encoder đọc câu hỏi và tài liệu cùng lúc nên xếp chuẩn hơn, nhưng nó phải chạy một lượt cho MỖI CẤP. Chạy nó trên 1,65 triệu tài liệu cho một câu hỏi là không khả thi. Chạy trên 1.000 ứng viên thì được.

Đổi lại: tầng hai KHÔNG thêm được tài liệu mới. Tài liệu tầng một bỏ sót thì mất luôn.

07b Xếp hạng lại GPU

Hai việc: nâng độ sâu 100 → 2.043 để nhóm “ngoài tầm với” co lại, rồi dùng cross-encoder sắp xếp lại.

Ở độ sâu 100 phải giành **29,4%** khoảng trống

Ở độ sâu 1.000 chỉ cần **20,9%**

Nâng độ sâu gần như không tốn thêm: việc đắt là dựng chỉ mục, và nó chạy một lần bất kể lấy 100 hay 2.000.

07c Mô hình ngữ nghĩa GPU

Biến văn bản thành vector, tìm vector gần nhau. Bắt được trường hợp câu hỏi viết “xe hơi” mà tài liệu viết “ô tô”.

Phải nhúng **1.654.055** tài liệu

Chi phí tỷ lệ với kích thước kho, không phải số câu hỏi

Trần kỳ vọng thấp — chỉ **0,7%** ngoài tầm từ khoá

Bước đắt nhất dự án, tính bằng giờ. Nếu nó giúp thì giúp ở chỗ xếp đúng hơn và ở chỗ hai mô hình sai khác nhau.

07d Kết hợp CPU

Gộp hai bảng xếp hạng bằng Reciprocal Rank Fusion, rồi chọn hệ thống đem nộp.

Ba điều kiện, không có điều kiện thứ tư: trả lời đủ mọi câu · cao nhất trên **fit** · hơn hệ thống đơn giản hơn một cách có ý nghĩa. Hoà thì chọn cái đơn giản hơn.

08 Chạy dev và đóng gói CPU

Chạy hệ thống đã chọn trên 519 câu **dev** — đúng một lần — rồi xuất file TREC và kiểm tra bằng công cụ ban tổ chức.

Notebook tự sinh **danh sách mô hình phải công khai** từ chính cấu hình hệ thống, nên không thể quên thứ gì đã thật sự chạy.

13 Thư mục `src/`

FILE	VIỆC	ĐIỂM ĐÁNG NỔI
<code>reteco.py</code> ~1.050 dòng	Nạp dữ liệu · cắt từ · chỉ mục · chấm điểm · chia tập · đo trần · kiểm định · bản sao	<code>RetrievalIndex</code> dựng chỉ mục một lần rồi đối giữa sáu mô hình bằng cách biến đổi lại mảng trọng số. Bước đầu là đếm từ, và nó chạy một lần cho cả sáu.
<code>pipeline.py</code> ~900 dòng	Khung ghép bốn chặng <code>retrieve</code> → <code>dedup</code> → <code>rerank</code> → <code>fuse</code> , chấm điểm, so sánh, xuất bài nộp	Mã bám theo chặng . Đối bộ xếp hạng lại thì mã bám chặng truy xuất không đổi, nên lấy thẳng từ đệm thay vì dựng lại chỉ mục 14 phút. Chặng <code>null</code> không vào mã bám.
<code>gpu_worker.py</code> ~330 dòng	Chạy phần GPU ở máy khác	Không import gì từ dự án. Máy GPU chỉ cần file này và thư mục gói việc. Ghi kết quả theo từng nhóm nên phiên bị ngắt thì chạy lại là làm tiếp.
<code>report.py</code> ~430 dòng	Sinh trang so sánh chạy local	Một file HTML tự chứa số liệu, không cần server. Từ chối vẽ biểu đồ so sánh giữa <code>dev</code> và <code>train</code> vì hai tập khác câu hỏi.
<code>run_candidates.py</code> ~150 dòng	Chạy hết ứng viên trong <code>systems/</code> rồi so sánh	Một ứng viên là một file JSON. Thêm ứng viên không cần sửa code.

14 Thuật ngữ

Phần tra cứu. Mọi thuật ngữ dùng ở trên đều có ở đây.

Khái niệm

Term	Đơn vị nhỏ nhất hệ thống làm việc. Ở đây là một từ : cắt theo “chuỗi chữ cái và chữ số liên tiếp”, hạ chữ thường. “Tesla’s Model 3” → <code>tesla</code> <code>s</code> <code>model</code> <code>3</code> .
Stopword	Từ ở khắp nơi nên không giúp phân biệt: <i>the, of, and</i> . Dự án dùng 173 từ . Riêng <code>the</code> chiếm 5,27% kho.
Chỉ mục ngược	Bảng tra ngược: “ từ này nằm trong những tài liệu nào ”. Nhanh hơn quét cả kho 2.158 lần .
Posting	Một cặp (từ, tài liệu) kèm số lần xuất hiện. Kho này có 216.320.198 posting.
Ứng viên	Danh sách từng một trả về cho từng hai. Hiện 100/câu, kế hoạch nâng lên 1.000.
Qrels	File đáp án. Ở bộ này mọi giá trị đều bằng 1 — nhị phân , không có mức độ.

Độ đo

nDCG@10 chính thức	Nhìn 10 kết quả đầu, tài liệu đúng nằm càng cao càng nhiều điểm, theo hệ số $1/\log_2(\text{vị trí}+1)$, rồi chia cho danh sách hoàn hảo. Luôn nằm giữa 0 và 1. <i>Vốn sinh ra cho nhân có nhiều mức độ; bộ này nhấn nhị phân nên phần “mức độ” không được dùng tới.</i>
P@10	Trong 10 kết quả đầu, bao nhiêu cái đúng. Bị chặn trên ở ~0,36 vì mỗi câu chỉ có trung bình 3,57 đáp án — con số tuyệt đối không nói lên nhiều, so giữa hai hệ thống thì có nghĩa.
R@k	Trong tổng số đáp án, bao nhiêu cái lọt vào k đầu. R@100 là độ đo quan trọng nhất cho từng một : đáp án không lọt vào thì bước sau không cứu được.
F1@10	Trung bình điều hoà của P@10 và R@10.
MAP	Với mỗi đáp án tìm được, tính độ chính xác tại vị trí của nó rồi lấy trung bình. Có xét thứ tự, định nghĩa sẵn cho nhấn nhị phân — nên nó kiểm chứng chéo với nDCG.
Trần xếp hạng lại	Điểm cao nhất một bộ xếp hạng lại hoàn hảo đạt được trên đúng danh sách hiện có. Là chặn trên chính xác, dùng để trả lời “bước tiếp theo có đáng làm không” <i>trước khi</i> bỏ công.
Macro average	Tính điểm từng nhóm rồi trung bình 13 số — không phải trung bình trên 1.211 câu.

Mô hình	Đặc điểm	Ưu điểm	Nhược điểm
1. Mô hình truyền thống	Chạy trên nền tảng máy chủ, người dùng truy cập qua trình duyệt.	Đơn giản, dễ triển khai, bảo mật cao.	Chi phí vận hành cao, khó mở rộng quy mô.
2. Mô hình đám mây	Ứng dụng được lưu trữ và chạy trên nền tảng đám mây.	Chi phí thấp, dễ mở rộng quy mô, bảo trì đơn giản.	Phụ thuộc vào nhà cung cấp dịch vụ đám mây.
3. Mô hình lai	Kết hợp giữa mô hình truyền thống và mô hình đám mây.	Flexibility cao, tối ưu chi phí.	Phức tạp trong quản lý và vận hành.

Boolean	Thoả biểu thức thì được, không thì loại. Không có thứ hạng. AND đòi mọi từ có mặt nên chỉ cần một từ hiếm là rỗng — vì vậy nó chỉ trả lời 141/1.211 câu.
TF-IDF · cosine	Mỗi tài liệu là một vector, mỗi chiều một từ, giá trị = số lần xuất hiện × độ hiếm. Độ tương đồng là cosine của góc giữa hai vector.
IDF	Độ hiếm của từ trong kho. Từ càng hiếm càng nặng ký.
BM25	Chuẩn thực tế của ngành từ thập niên 1990. Ba thành phần: IDF · bảo hoà tần suất (tham số k₁) · chuẩn hoá độ dài (tham số b). Họ BM nằm trong cùng một lưới: b=0 là BM15, b=1 là BM11, ở giữa là BM25.
Query Likelihood	Đảo ngược câu hỏi: coi mỗi tài liệu là một cỗ máy sinh chữ, xác suất nó tình cờ sinh ra đúng câu hỏi này là bao nhiêu? <i>Phải làm trơn</i> : ước lượng thẳng thì một từ tài liệu không chứa cho xác suất 0, nhân vào làm cả tích bằng 0. Cách chữa là trộn thống kê tài liệu với thống kê cả kho.
Cross-encoder	Đọc câu hỏi và tài liệu cùng một lượt nên bắt được quan hệ giữa chữ hai bên. Xếp chuẩn hơn nhiều, nhưng phải chạy một lượt cho <i>mỗi cặp</i> .
Bi-encoder	Biến mỗi tài liệu thành vector <i>độc lập</i> với câu hỏi. Nhúng trước được cả kho nên tìm nhanh, nhưng kém chuẩn hơn cross-encoder.
RRF	$\text{điểm}(d) = \sum 1 / (k + \text{thứ hạng})$. Cộng thứ hạng chứ không cộng điểm, vì điểm hai mô hình không cùng thang đo.

Phương pháp

Ablation	Bỏ một thành phần rồi đo xem mất bao nhiêu. Cách duy nhất biết phần nào thật sự làm nên kết quả.
Bootstrap phân tầng	Lấy mẫu có hoàn lại từ chính các câu truy vấn hàng nghìn lần, xem 95% các lần rơi vào khoảng nào. Khoảng chứa 0 = hoà , bất kể chênh lệch là bao nhiêu. “Phân tầng theo nhóm” = lấy mẫu trong từng nhóm rồi mới trung bình, cho khớp cách chấm.
fit / check / dev	1.211 câu train chia fit 969 / check 242 . Mọi lựa chọn quyết định trên fit; check chỉ đọc <i>sau khi</i> đã chọn. dev 519 câu chạy đúng một lần ở cuối.
Thiên lệch do chọn cái tốt nhất	Thử nhiều phương án rồi giữ cái cao nhất thì cái được giữ có điểm cao hơn mức thật. Mô phỏng 20 phương án ngang tài: cái thắng bị thổi phồng +0,0280 .
Cổng kiểm tra	Phép so bắt buộc khớp trước khi tin số liệu. Dự án có hai: BM25 tự viết phải cho thứ tự <i>giống hệt</i> bản ban tổ chức, và khung mô-đun phải cho lại đúng con số notebook trước (sai số 0,00e+00).

Tham số			
THAM SỐ	CỦA	ĐIỀU KHIỂN	ĐANG DÙNG
<code>k1</code>	BM25	Bảo hoà tần suất. <code>→0</code> : lặp bao nhiêu cũng như nhau. <code>→∞</code> : điểm tỷ lệ bằng với số lần lặp.	2,0
<code>b</code>	BM25	Trùng phạt tài liệu dài. <code>0</code> = không, <code>1</code> = tối đa.	0,9
<code>mu</code>	Query Likelihood	Lượng chứng cứ mượn từ kho khi làm trơn: mỗi tài liệu được cộng <code>μ</code> chữ tương tự.	1.000
<code>depth</code>	tầng một	Lấy về bao nhiêu ứng viên. Sâu hơn thì trần cao hơn, gần như không tốn thêm.	100 → 2.043
<code>expand</code>	khử trùng lặp	Tối đa bao nhiêu mã của cùng một nội dung được xuất hiện. 1 = tin vào việc gộp.	2
<code>k</code>	RRF	Làm dịu đầu bằng. <code>k</code> lớn thì muốn lên đầu phải được nhiều bằng cùng ứng hộ.	sẽ đo ở 07d
<code>query_form</code>	câu hỏi	<code>title_weighted</code> = lặp tiêu đề 3 lần rồi nối cả bài.	title_weighted
<code>tokenizer</code>	cắt từ	<code>nohtml_stop</code> = bỏ chữ của thẻ HTML và bỏ hư từ.	nohtml_stop

15 Giả thuyết đã bị số liệu bác bỏ

Ghi lại cả phần sai là chủ ý — nó cho thấy quy trình có tự kiểm tra, và giúp người đọc sau không mất công thử lại.

GIẢ THUYẾT	SỐ LIỆU CHO THẤY	
Bỏ hư từ và cắt đuôi kiểu Anserini sẽ giúp	Làm tệ hơn cả hệ thống cơ sở	03a
Bắt chước Lucene sẽ đóng khoảng cách tới 0,0879	Không đóng được chút nào — 0,0706	03a
Bỏ thẻ HTML là phần thắng lớn nhất	Chỉ đáng +0,0003 khi đã có danh sách hư từ	03a
BM25 đã chạm trần, phải cần mô hình ngữ nghĩa	99,3% đáp án vẫn trong tầm từ khoá	03b
BM25 hơn hẳn TF-IDF	Hoà , và TF-IDF thắng nếu đổi cách cắt từ	04
BM25 và TF-IDF là hai mô hình cổ điển mạnh nhất	Query Likelihood thắng cả hai , chênh lệch thật	04
IDF là thành phần quan trọng nhất	Chuẩn hoá độ dài đáng gấp 5 lần IDF	05
Gộp hai mô hình hơn hẳn lấy sâu gấp đôi	Chênh 0,0022 , coi như hoà	05

Một kết luận đã phải sửa lại

Notebook **04** kết luận IDF chiếm gần như toàn bộ điểm số, dựa trên `tf` = 0,0021 so với `tfidf` = 0,1407.

Phép so đó đổi hai thứ cùng lúc — `tf` vừa không có IDF, vừa không có chuẩn hoá độ dài. Notebook **05** tách riêng ra.

Sai sót này vì phạm đúng nguyên tắc “chỉ đổi một thứ mỗi lần” của chính dự án. Nó được ghi lại thay vì xoá đi.

<code>tf</code> — không IDF, không chuẩn hoá	0,0021
<code>tfidf_no_idf</code> — không IDF, có chuẩn hoá	0,1186
<code>tfidf</code> — có cả hai	0,1407

Chuẩn hoá độ dài **+0,1165** · IDF **+0,0221** — gấp 5 lần.

Mười nguyên tắc làm việc

1. **Phát triển trên `train . dev`** chỉ chạy một lần ở bước cuối.
2. **Chỉ đổi một thứ mỗi lần.**
3. **Kiểm định theo nhóm.**
4. **Luôn báo cáo số câu trả lời được.**
5. **Hoà thì chọn cái đơn giản hơn.**
6. **Không kết luận từ một nhóm.** Trần giữa các nhóm chênh 3,1 lần.

7. Ghi lại cả giả thuyết sai.
8. Không dùng rò rỉ mã tài liệu.
9. Đo trần trước khi đầu tư.
10. Chỉ truy hồi trong kho đã cho, và công khai mọi mô hình đã dùng.

Nguồn

datascienceuibk.github.io/RETECO ·
huggingface.co/datasets/DataScience-UIBK/RETECO-SemEval2027 ·
github.com/DataScienceUIBK/RETECO

Thư viện

numpy · scipy · matplotlib. Phần GPU (chưa chạy) sẽ dùng thêm torch và sentence-transformers.

Dữ liệu ngoài

Không dùng nguồn nào.